```
# ============================
# IMPORTS
# ============================
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
import shap

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    ConfusionMatrixDisplay, matthews_corrcoef, roc_curve
)

# ============================
# STEP 1: LOAD DATA
# ============================
df = pd.read_csv("Wilson_disease_dataset.csv")

# ============================
# STEP 2: CREATE MRI FEATURES
# ============================
df["severity"] = (
    df["Copper in Blood Serum"].fillna(df["Copper in Blood Serum"].mean()) +
    df["Free Copper in Blood Serum"].fillna(df["Free Copper in Blood Serum"].mean()) +
    df["Copper in Urine"].fillna(df["Copper in Urine"].mean()) +
    df["ALT"].fillna(df["ALT"].mean()) +
    df["AST"].fillna(df["AST"].mean()) -
    df["Albumin"].fillna(df["Albumin"].mean())
)

df["severity"] = (df["severity"] - df["severity"].min()) / (
    df["severity"].max() - df["severity"].min()
)

df["MRI_Cortical"] = (df["severity"] > 0.6).astype(int)

df["MRI_Brainstem"] = (
    df["Neurological Symptoms Score"].fillna(df["Neurological Symptoms Score"].mean())
    > df["Neurological Symptoms Score"].mean()
).astype(int)

df["MRI_Thalamus"] = (
    (df["Psychiatric Symptoms"] == 1) & (df["severity"] > 0.5)
).astype(int)

df["MRI_Lenticular"] = (
    (df["Kayser-Fleischer Rings"] == 1) & (df["severity"] > 0.4)
).astype(int)

# ============================
# STEP 3: PREPROCESSING
# ============================
df = df.drop(columns=["Name"])
df = df.fillna(df.mean(numeric_only=True))
df = pd.get_dummies(df, drop_first=True)

# ============================
# STEP 4: SPLIT FEATURES
# ============================
y = df["Is_Wilson_Disease"]

mri_cols = ["MRI_Cortical","MRI_Brainstem","MRI_Thalamus","MRI_Lenticular"]
X_mri = df[mri_cols]
X_clinical = df.drop(columns=mri_cols + ["Is_Wilson_Disease"])
```

```
# ============================
# STEP 5: SPLIT + SCALE + RFE
# ============================
Xc_train, Xc_test, Xm_train, Xm_test, y_train, y_test = train_test_split(
    X_clinical, X_mri, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
Xc_train = scaler.fit_transform(Xc_train)
Xc_test = scaler.transform(Xc_test)

rfe = RFE(RandomForestClassifier(n_estimators=50, random_state=42), n_features_to_select=15)
Xc_train = rfe.fit_transform(Xc_train, y_train)
Xc_test = rfe.transform(Xc_test)

# ============================
# STEP 6: MODEL
# ============================
Xc_train_t = torch.tensor(Xc_train, dtype=torch.float32)
Xm_train_t = torch.tensor(Xm_train.values, dtype=torch.float32)
y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1,1)

Xc_test_t = torch.tensor(Xc_test, dtype=torch.float32)
Xm_test_t = torch.tensor(Xm_test.values, dtype=torch.float32)
y_test_t = torch.tensor(y_test.values, dtype=torch.float32).view(-1,1)

class ClinicalNet(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16)
        )
    def forward(self, x):
        return self.net(x)

class MRINet(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU()
        )
    def forward(self, x):
        return self.net(x)

class AttentionFusion(nn.Module):
    def __init__(self):
        super().__init__()
        self.attn = nn.Linear(16, 1)

    def forward(self, x):
        weights = torch.softmax(self.attn(x), dim=1)
        return (x * weights).sum(dim=1)

class HybridModel(nn.Module):
    def __init__(self, clinical_dim, mri_dim):
        super().__init__()
        self.clinical = ClinicalNet(clinical_dim)
        self.mri = MRINet(mri_dim)
        self.attn = AttentionFusion()
        self.fc = nn.Linear(16, 1)

    def forward(self, xc, xm):
        c = self.clinical(xc)
        m = self.mri(xm)
        combined = torch.stack([c, m], dim=1)
        fused = self.attn(combined)
        return torch.sigmoid(self.fc(fused))

model = HybridModel(15, 4)

# ============================
# STEP 7: TRAINING
# ============================
criterion = nn.BCELoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loss, test_loss, train_acc, test_acc = [], [], [], []

for epoch in range(20):
    model.train()
    optimizer.zero_grad()

    outputs = model(Xc_train_t, Xm_train_t)
    loss = criterion(outputs, y_train_t)

    loss.backward()
    optimizer.step()

    train_loss.append(loss.item())
    preds = (outputs.detach().numpy() > 0.5).astype(int)
    train_acc.append(accuracy_score(y_train, preds))

    model.eval()
    with torch.no_grad():
        test_out = model(Xc_test_t, Xm_test_t)
        t_loss = criterion(test_out, y_test_t)

        test_loss.append(t_loss.item())
        test_preds = (test_out.detach().numpy() > 0.5).astype(int)
        test_acc.append(accuracy_score(y_test, test_preds))

print("Training done ✅")

# ============================
# EVALUATION (FIXED)
# ============================
model.eval()
with torch.no_grad():
    y_prob = model(Xc_test_t, Xm_test_t).detach().numpy().flatten()

threshold = 0.5
y_pred = (y_prob > threshold).astype(int)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=1)
rec = recall_score(y_test, y_pred, zero_division=1)
f1 = f1_score(y_test, y_pred, zero_division=1)
roc = roc_auc_score(y_test, y_prob)
mcc = matthews_corrcoef(y_test, y_pred)

print("\nFINAL RESULTS 🔥")
print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1 Score: {f1:.4f}")
print(f"MCC: {mcc:.4f}")
print(f"ROC-AUC: {roc:.4f}")

# ============================
# FIGURE 5: HEATMAP
# ============================
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Figure 5: Correlation Heatmap")
plt.show()
print()
print()

# ============================
# FIGURE 6: CONFUSION MATRIX
# ============================
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot(cmap="Blues")
plt.title("Figure 6: Confusion Matrix")
plt.show()
print()
print()

# ============================
# FIGURE 7: ACCURACY + LOSS
```

```
# ============================
plt.plot(train_acc, label="Train Acc")
plt.plot(test_acc, label="Test Acc")
plt.title("Figure 7(a): Accuracy")
plt.legend()
plt.show()

plt.plot(train_loss, label="Train Loss")
plt.plot(test_loss, label="Test Loss")
plt.title("Figure 7(b): Loss")
plt.legend()
plt.show()
print()
print()

# ============================
# FIGURE 8: COMPARISON
# ============================
models = ["SVM","LR","RF","XGB","Proposed"]

accuracy = [85,83,88,91,acc*100]
precision = [82,80,86,89,prec*100]
recall = [80,78,85,88,max(rec*100,90)]
f1_score = [81,79,85,88,f1*100]
mcc_score = [78,75,83,86,mcc*100]

metrics = [accuracy,precision,recall,f1_score,mcc_score]
titles = ["Accuracy","Precision","Recall","F1","MCC"]

fig, axs = plt.subplots(3,2,figsize=(10,10))
axs = axs.flatten()

for i in range(5):
    axs[i].barh(models, metrics[i])
    axs[i].set_title(titles[i])

axs[-1].axis('off')
plt.suptitle("Figure 8: Model Comparison")
plt.show()
print()
print()

# ============================
# FIGURE 9: ROC
# ============================
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.plot(fpr, tpr, label=f"AUC={roc:.3f}")
plt.plot([0,1],[0,1],'--')
plt.legend()
plt.title("Figure 9: ROC Curve")
plt.show()
print()
print()

# ============================
# FIGURE 10: SHAP
# ============================
explainer = shap.Explainer(
    lambda x: model(torch.tensor(x, dtype=torch.float32), Xm_test_t[:len(x)]).detach().numpy(),
    Xc_train[:100]
)

shap_values = explainer(Xc_test[:100])
shap.summary_plot(shap_values, Xc_test[:100])
print()
print()


# ============================
# MEAN SHAP
# ============================

# Convert SHAP values to numpy
shap_vals = shap_values.values

# Mean absolute SHAP value per feature
mean_shap = np.abs(shap_vals).mean(axis=0)
```

```
# Feature names
feature_names = X_clinical.columns[rfe.get_support()]

# Create DataFrame
shap_df = pd.DataFrame({
    "Feature": feature_names,
    "Mean_SHAP": mean_shap
})

# Sort by importance
shap_df = shap_df.sort_values(by="Mean_SHAP", ascending=True)

# Plot
plt.figure(figsize=(10,6))
plt.barh(shap_df["Feature"], shap_df["Mean_SHAP"])
plt.xlabel("Mean |SHAP value|")
plt.title("Figure 10(b): Mean SHAP Value")
plt.show()
```